

# Final Report

## **Predicting LLM Response Quality Scores:**

*A Low-Cost, Interpretable Approach to Real-Time LLM Quality Monitoring*

Ethan Kwon • Henry Kim • Jihun Jeong

Group 15 | CMU Tepper School of Business

Machine Learning for Business | April 2026

# 1. Business Problem & Motivation

As large language models (LLMs) are rapidly deployed across enterprise applications — from customer service automation to financial report generation — ensuring the quality of model outputs has become a critical operational challenge. Organizations that rely on LLMs in production need to continuously monitor whether their models are generating useful, high-quality responses. Without this, degradation in model behavior can go undetected until it affects end users or regulatory compliance.

Today, organizations rely on two primary evaluation methods, both of which are severely limited:

- **Human annotation:** Domain experts manually score LLM responses for accuracy, helpfulness, and coherence. This approach is expensive — Meta reportedly spent over \$8 million on preference data for LLaMA 2 alone — and does not scale to production monitoring where millions of responses may need to be evaluated daily.
- **LLM-as-Judge:** A secondary, more powerful LLM (typically GPT-4) automatically scores the responses of the primary model. While faster than human annotation, this approach incurs significant API costs and introduces systematic biases that vary depending on which judge model is used, making results difficult to compare across evaluations.

Our proposed solution is to build a supervised regression model that predicts human preference quality scores for LLM responses based on 17 measurable textual features — without requiring a human evaluator or a secondary LLM at inference time. This delivers three distinct forms of business value:

- **Cost savings:** Eliminates dependence on expensive human labeling pipelines and API-based judge models. Once trained, inference costs are negligible.
- **Speed:** Enables near-real-time quality monitoring. Scores are generated in milliseconds per response, making continuous production monitoring feasible.
- **Governance:** Provides an audit-ready, automated scoring mechanism aligned with AI risk management frameworks such as NIST AI RMF and SR 11-7. AI governance teams can use the system to track quality trends over time and flag regressions.

The primary end users of this system are LLM product teams, model evaluation engineers, and AI governance teams at organizations that deploy LLMs in production. A successful system produces quality scores that correlate meaningfully with human judgments, enabling these teams to monitor and improve their models without incurring the cost and latency of human annotation.

## 2. Data

### 2.1 Dataset: LMSYS Chatbot Arena Conversations

We used the LMSYS Chatbot Arena Conversations dataset, released by Zheng et al. (2023) alongside the Chatbot Arena platform and publicly available on Hugging Face. The dataset was collected from a crowdsourced platform where human users compare two LLM responses side-by-side and vote for the one they prefer.

- **Size:** 33,000 crowd-sourced conversations
- **Coverage:** Responses from 20+ LLMs including GPT-4, Claude-v1, LLaMA, Vicuna, and others
- **Labels:** Human pairwise preference votes: model\_a wins / model\_b wins / tie
- **Additional fields:** Model names, conversation turns, language tags
- **Access:** [huggingface.co/datasets/lmsys/chatbot\\_arena\\_conversations](https://huggingface.co/datasets/lmsys/chatbot_arena_conversations)

### 2.2 Target Variable Construction

The original dataset consists of pairwise comparisons — each row represents one comparison between two model responses with a single human vote. To enable supervised regression, we transformed this structure into a row-level dataset where each record represents one response instance with a continuous quality score.

The transformation proceeded as follows:

- Each pairwise comparison was unpivoted into two rows — one for response A and one for response B.
- The derived target variable, win\_score, was assigned as: **1.0 for a win, 0.5 for a tie, and 0.0 for a loss.**
- This produced a final dataset of 66,000 row-level response records suitable for regression.

The target distribution has a notable characteristic: approximately 48% of votes are ties, with roughly 26% each for model\_a wins and model\_b wins. This discrete and tie-heavy distribution limits the achievable RMSE improvement and motivates the use of Spearman rank correlation as the primary evaluation metric (discussed in Section 4).

### 2.3 Notable Data Characteristics

- **Class imbalance equivalent:** The tie-heavy target distribution (~48% ties) makes it difficult to differentiate quality at the margins. All models cluster predictions near 0.5.
- **Multi-lingual:** The dataset is English-dominant but contains conversations in multiple languages. Non-English responses may have different textual feature distributions.
- **Discrete target:** The three-valued win\_score (0, 0.5, 1) creates a discretized regression problem. The vertical scatter in predicted vs. actual plots reflects this structure.
- **No missing values:** All records with empty or very short responses (fewer than 10 characters) were filtered out during preprocessing, leaving a clean dataset.

## 3. Methodology

### 3.1 Feature Engineering

We extracted 17 interpretable features from each prompt-response pair, organized into four categories. All features are computed deterministically from raw text using standard Python libraries (textstat, basic string operations), requiring no external API calls or model inference. This makes the feature extraction pipeline fast, fully reproducible, and free of operational cost.

| Category            | Features                                                                         | Rationale                              |
|---------------------|----------------------------------------------------------------------------------|----------------------------------------|
| Lexical (6)         | word_count, char_len, sentence_count, avg_word_len, avg_sent_len, TTR            | Verbosity and vocabulary richness      |
| Structural (6)      | has_bullets, has_numbered, has_code, has_headers, newline_count, paragraph_count | Formatting quality and organization    |
| Readability (3)     | Flesch Reading Ease, Flesch-Kincaid Grade, Gunning Fog                           | Complexity and clarity proxies         |
| Prompt-relative (2) | prompt_word_count, response-to-prompt ratio                                      | Contextualizes length relative to task |

The feature engineering choices were informed by prior work on RLHF reward models, which consistently show that response length, structure, and formatting are strong predictors of human preference. The readability features (Flesch, Flesch-Kincaid, Gunning Fog) were included as proxies for response complexity, while the prompt-relative ratio contextualizes length relative to the difficulty of the task.

### 3.2 Models Trained

#### Ridge Regression (Baseline)

Ridge Regression is a regularized linear model that fits a weighted sum of input features to the target variable with L2 regularization ( $\alpha = 1.0$ ). It was chosen as the baseline because it establishes the minimum performance that any more complex model should exceed, and its coefficients are directly interpretable. Features were standardized using StandardScaler (fitted on the training set only) to ensure fair regularization across features of different scales.

#### Random Forest Regressor

Random Forest is an ensemble of decision trees trained on bootstrap samples of the training data, with each split considering a random subset of features. Configuration: `n_estimators=200`, `max_depth=8`, `min_samples_leaf=10`, `random_state=42`. By averaging predictions across many diverse trees, it captures non-linear relationships between features and win-rate scores while reducing variance through bagging.

#### XGBoost Regressor

XGBoost builds trees sequentially, with each new tree correcting the residual errors of previous ones. Configuration: `n_estimators=300`, `max_depth=5`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `reg_alpha=0.1`. Both L1 and implicit L2 regularization are applied, along with row and column subsampling per tree to reduce overfitting.

### 3.3 Training & Evaluation Best Practices

#### Data Splitting

We applied a three-way split to ensure unbiased evaluation and prevent data leakage: 80% training, 10% validation, 10% test. All splits were performed with a fixed random seed (`random_state=42`) for reproducibility. The test set was used only once, at final evaluation, to report all performance metrics.

### **5-Fold Cross-Validation**

In addition to the holdout test set, we applied 5-fold cross-validation on the training set to obtain a more robust estimate of generalization performance (CV RMSE mean  $\pm$  std). This guards against the risk that a single train/test split produces an unrepresentative estimate of model quality.

### **Feature Standardization**

For Ridge Regression, features were standardized using `StandardScaler` fitted exclusively on the training set and then applied to validation and test sets. This prevents data leakage from the test distribution. Tree-based models (Random Forest, XGBoost) are scale-invariant and do not require this preprocessing step.

## 4. Results & Model Selection

### 4.1 Evaluation Metrics

We selected three complementary metrics to evaluate model performance, each addressing a distinct aspect of quality relevant to our business problem:

- **RMSE (Root Mean Squared Error):** Measures the average magnitude of prediction error in the same units as the target variable (win-rate score, 0–1). Penalizes large errors more heavily due to squaring. Business relevance: large mispredictions are more costly than small ones in a quality monitoring system.
- **MAE (Mean Absolute Error):** Measures the average absolute difference between predicted and actual scores. More robust to outliers than RMSE and provides an interpretable 'average error' in win-rate units.
- **Spearman Rank Correlation ( $\rho$ ):** Measures the monotonic relationship between predicted and actual win-rate rankings without assuming linearity. This is the most important metric for our use case because the core business value is ranking LLM responses by quality, not predicting exact scores. A model with high Spearman  $\rho$  correctly orders responses even if its absolute predictions are slightly off. This metric directly mirrors the ranking objective underlying human preference data.

### 4.2 Model Comparison

| Model                       | Test RMSE ↓ | Test MAE ↓ | Spearman $\rho$ ↑ | CV RMSE (5-fold) |
|-----------------------------|-------------|------------|-------------------|------------------|
| Ridge Regression (baseline) | 0.4150      | 0.3600     | 0.1865            | 0.4126 ± 0.0012  |
| Random Forest ♦ selected    | 0.4083      | 0.3568     | 0.2416 ← best     | 0.4087 ± 0.0007  |
| XGBoost                     | 0.4088      | 0.3567     | 0.2337            | 0.4091 ± 0.0003  |

### 4.3 Final Model: Random Forest

We selected Random Forest as our final production model based on four factors:

- **Best ranking performance:** Random Forest achieves the highest Spearman  $\rho$  (0.2416 vs. 0.2337 for XGBoost and 0.1865 for Ridge). Since our business goal is to rank responses by quality — not predict exact scores — Spearman correlation is the primary selection criterion.
- **Most stable generalization:** CV RMSE of  $0.4087 \pm 0.0007$  shows the tightest variance across folds, indicating reliable out-of-sample performance.
- **Interpretability:** Random Forest provides native feature importance scores. `paragraph_count` (0.334), `has_bullets` (0.093), and `ttr` (0.056) are the top three predictors, giving users explainable quality signals that align with intuition about what makes a good response.
- **Latency:** Inference on 17 lightweight textual features is near-instantaneous, suitable for real-time use without GPU or API dependency.

### 4.4 Interpretation of Results

All three models show similar RMSE (~0.41), which is consistent with the discrete and tie-heavy target distribution. When actual values cluster at 0, 0.5, and 1, any model will exhibit vertical scatter in predicted vs. actual plots — this is a property of the data structure, not a failure of the models. The more meaningful differentiation comes from Spearman correlation, where Random Forest's non-linear ensemble approach captures useful structure that the linear Ridge model cannot.

The feature importance results are intuitively interpretable: `paragraph_count` is the dominant predictor by a wide margin, suggesting that longer, well-structured responses with clear paragraph organization are consistently preferred by human evaluators. `has_bullets` and `sentence_count` confirm that formatting richness and structural variety matter more than readability scores (Flesch, Gunning Fog), which rank near the bottom of the importance ranking.

Performance remains modest overall. This is expected at this stage — the current feature set captures surface-level textual signals without any semantic understanding. Adding BERTScore, perplexity scores, and toxicity signals is the clearest path to meaningfully improving Spearman  $\rho$  in future iterations.

## 5. Cloud Architecture

### 5.1 AWS Services

| AWS Service        | Role                                                  | Status    |
|--------------------|-------------------------------------------------------|-----------|
| Amazon S3          | Stores raw data and trained model artifact (.pkl)     | ✓ Live    |
| SageMaker Training | Managed training job with scikit-learn container      | ✓ Used    |
| EC2 (t2.micro)     | Hosts Streamlit app; externally accessible port 8501  | ✓ Live    |
| Amazon RDS         | PostgreSQL for prediction logging (planned iteration) | ⚠ Planned |

### 5.2 Architecture Description

The system follows two distinct operational paths:

- **Training path:** Raw data and training scripts are stored in Amazon S3. A SageMaker Training Job runs the scikit-learn container to train the Random Forest model on the full training dataset. The trained model artifact (random\_forest\_model.pkl) is saved back to S3 upon job completion.
- **Inference path:** A user enters a prompt and LLM response text into the Streamlit web application running on an EC2 t2.micro instance. The application extracts 17 textual features in real time, loads the trained model artifact, generates a win-rate score prediction via Random Forest inference, and displays the result — including a quality grade and feature breakdown — immediately in the browser. The EC2 instance is externally accessible on port 8501 via a properly configured Security Group inbound rule.

### 5.3 Architecture Diagram

The following describes the system architecture (see accompanying diagram):

- S3 → SageMaker Training Job → S3 (model artifact) [Training Path]
- User Browser → EC2 Streamlit (port 8501) → RF Model Inference → Score Display [Inference Path]

Amazon RDS (PostgreSQL) for prediction logging is planned for a future iteration. In the current implementation, scores are displayed in the UI only and are not persisted.



## 6. User Interface

### 6.1 Interface Description

We built a Streamlit web application that provides a simple, functional interface for real-time LLM response quality scoring. The application is deployed on EC2 and is externally accessible at the EC2 public IP on port 8501.

### 6.2 Inputs

- **Prompt:** The original user question or instruction given to the LLM, entered as free text.
- **LLM Response:** The generated text to be evaluated, pasted into the response text area.

### 6.3 Outputs

- **Win-Rate Score (0–1):** The predicted probability that human evaluators would prefer this response over a randomly selected alternative. Displayed as a numeric score with a progress bar.
- **Quality Grade:**  High Quality ( $\geq 0.70$ ) — response is likely to be preferred.  Moderate Quality (0.45–0.70) — response may win some comparisons but has room for improvement.  Low Quality ( $< 0.45$ ) — response is unlikely to be preferred over a well-structured alternative.
- **Feature Breakdown (expandable):** All 17 extracted feature values are shown in an expandable panel, allowing users to understand what textual signals drove the score. This enables actionable feedback — for example, a low score with low paragraph\_count suggests that improving response structure would increase quality.

### 6.4 Sidebar

A persistent sidebar displays model metadata: model type, target variable definition, feature categories, Spearman  $\rho$ , test RMSE, and dataset source. This contextualizes the score for users who need to understand the model's basis and limitations.

Deploy

About

Model: Random Forest Regressor  
Target: Win-rate score (0 = loss, 0.5 = tie, 1.0 = win)  
Features: 17 textual features  
Spearman  $\rho$ : 0.2416 (best on test set)  
Test RMSE: 0.4083  
Dataset: LMSYS Chatbot Arena (33K conversations → 66K rows)

CMU Tepper MSBA  
Machine Learning for Business  
Group 15 — 2025

Feature categories:

- Lexical (6 features)
- Structural (6 features)
- Readability (3 features)
- Prompt-relative (2 features)

LLM Response Quality Scorer

Predict the human preference win-rate score of an LLM response using interpretable textual features — no secondary LLM required.

Model loaded from local file

Prompt (user question)

LLM Response

Enter the prompt / question that was given to the LLM...

Paste the LLM response to evaluate here...

Score This Response

## 7. Reflections & Lessons Learned

### 7.1 What Worked

- **Regression framing of pairwise preference data:** Converting binary pairwise votes into a continuous win\_score regression target unlocked richer training signal than a simple classification approach. The 0.5 tie score provides gradient information at the center of the distribution that a binary win/loss label would discard.
- **Lightweight, interpretable features:** The 17 textual features are deterministic, fast to compute, and require no external dependencies. This made the inference pipeline fast enough for real-time use and eliminated API cost and latency concerns that would arise from using embedding models or external scoring APIs.
- **Random Forest's native interpretability:** The built-in feature importance mechanism provided a clear, intuitive explanation of model behavior. The dominance of paragraph\_count and has\_bullets confirmed our hypothesis that structural formatting is the strongest signal of response quality in the LMSYS dataset.
- **Spearman correlation as the primary metric:** Recognizing early that ranking — not exact score prediction — is the core business objective led us to optimize for Spearman  $\rho$  rather than RMSE. This framing guided our model selection toward Random Forest over XGBoost, despite the two models having nearly identical RMSE.
- **EC2 + Streamlit deployment:** Once the AWS Academy Security Group configuration was correctly set (port 8501 inbound rule), the Streamlit deployment was straightforward and produced a functional, live-accessible application without the complexity of SageMaker Endpoints.

### 7.2 What Was Harder Than Expected

- **Discrete target distribution:** The three-valued win\_score (0, 0.5, 1) creates a fundamentally discretized regression problem. All models exhibit RMSE plateau behavior around 0.41 because predictions naturally cluster near the three target values. This was a data structure constraint, not a modeling failure, but it took time to diagnose.
- **AWS Academy Security Group configuration:** Initial EC2 deployment was blocked by an incorrectly configured Security Group that did not allow inbound TCP traffic on port 8501. This caused several hours of debugging before the root cause was identified. The lesson: always verify Security Group inbound rules immediately after launching an EC2 instance.
- **Tie-heavy target making margin differentiation difficult:** With ~48% of votes being ties, the model has limited signal to learn from at the quality margins — responses that are 'somewhat better' than average are difficult to distinguish from ties. This is an inherent limitation of the pairwise comparison setup and would require either more granular human ratings or semantic features to address.
- **Feature engineering scope vs. time:** We planned to add BERTScore and perplexity features but did not have sufficient time to integrate them before the demo deadline. These remain the highest-priority next steps.

### 7.3 What We Would Do Differently

- **Add semantic features earlier:** BERTScore and perplexity should be part of the initial feature set, not an afterthought. They are the most likely path to meaningfully improving Spearman  $\rho$  beyond the ~0.24 achieved with surface-level features.
- **Use a continuous preference score target:** The binary/tie structure of the LMSYS dataset inherently limits regression performance. A dataset with continuous quality ratings (e.g., 1–5 scale) would provide a richer training signal and likely produce better-calibrated predictions.

- **Implement RDS logging from the start:** Persisting predictions to a database would enable longitudinal quality monitoring — the core use case for AI governance teams — rather than one-off scoring. This should be part of the initial architecture rather than a planned next step.
- **Rehearse EC2 configuration earlier:** Security Group and networking configuration should be tested immediately when the instance is launched, not the day before the demo.

## 7.4 What We Would Build Next

- **Semantic feature integration:** Add BERTScore F1, GPT-2 perplexity, and Perspective API toxicity scores as additional features. These are expected to provide substantial improvement in Spearman  $\rho$ .
- **DeBERTa reward model:** Fine-tune a DeBERTa-based reward model on the LMSYS dataset as a stronger end-to-end baseline, following the RLHF literature (Ouyang et al., 2022).
- **RDS prediction logging:** Integrate Amazon RDS to persist all scoring requests and results, enabling quality trend dashboards and model drift detection.
- **Hyperparameter tuning:** Run GridSearchCV or Optuna on Random Forest and XGBoost hyperparameters to squeeze additional performance from the current feature set.

## 8. References

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS 2023)*.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askeel, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS 2022)*.

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2019). BERTScore: Evaluating Text Generation with BERT. *International Conference on Learning Representations (ICLR 2020)*.

Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. (2020). Learning to Summarize from Human Feedback. *Advances in Neural Information Processing Systems (NeurIPS 2020)*.

LMSYS Chatbot Arena Conversations Dataset. Hugging Face.  
[https://huggingface.co/datasets/lmsys/chatbot\\_arena\\_conversations](https://huggingface.co/datasets/lmsys/chatbot_arena_conversations)

---

*Code repository: [github.com/ethankwon22/MLBA\\_Group10](https://github.com/ethankwon22/MLBA_Group10) | Dataset: LMSYS Chatbot Arena  
([huggingface.co/datasets/lmsys/chatbot\\_arena\\_conversations](https://huggingface.co/datasets/lmsys/chatbot_arena_conversations)) | CMU Tepper MSBA 2026*